Trabalho Prático 2: Exercício 3

Neste trabalho vamos construir um SFOTS que descreve o programa que calcula a multiplicação de dois inteiros, utilizando BitVec de tamanho n, verificar se a multiplicação está correta, se o estado de Error pode ser alcançado com determinadas condições iniciais e se o SFOTS termina para quaisquer valores iniciais de a e b.

In [3]:
```
from pysmt.shortcuts import *
from z3 import *
```

A função declare cria as variáveis de cada estado, pc indica o nodo, a e b são os valores de entrada e x, y e z são as variáveis usadas na multiplicação de inteiros.

In [4]:
```
def declare(i, n):
    state = {}
    state['pc'] = BitVec(f'pc_{i}', n)
    state['a']  = BitVec(f'a_{i}', n)
    state['b']  = BitVec(f'b_{i}', n)
    state['x']  = BitVec(f'x_{i}', n)
    state['y']  = BitVec(f'y_{i}', n)
    state['z']  = BitVec(f'z_{i}', n)
    return state
```

A função init define o estado inicial. x recebe a, y recebe b e z começa igual a 0. O pc começa em 0, que corresponde ao nodo init.

In [5]:
```
def init(curr, n):
    return And(
        curr['pc'] == BitVecVal(0, n),
        curr['z'] == BitVecVal(0, n),
        curr['x'] == curr['a'],
        curr['y'] == curr['b'],
    )
```

A função trans define as transições entre nodos e as alterações das variáveis por transição.

In [6]:
```
def trans(curr, prox, n):
    a_b_constantes = And(curr['a'] == prox['a'], curr['b'] == prox['b'])

    shift_overflow = Extract(n-1, n-1, curr['x']) == BitVecVal(1, 1)
    soma_overflow = ULT(curr['z'] + curr['x'], curr['z'])

    # init = 0, skip = 1, left = 2, rigth = 3, stop = 4, error = 5
    return And(
        a_b_constantes,
        Or(
            # init → skip
            And(
                curr['pc'] == BitVecVal(0 ,n),
                prox['pc'] == BitVecVal(1 ,n),
```

```
            prox['x']  == curr['x'],
            prox['y']  == curr['y'],
            prox['z']  == curr['z']
        ),
        # skip → left
        And(
            curr['pc'] == BitVecVal(1 ,n),
            curr['y'] ≠ BitVecVal(0, n),
            URem(curr['y'], BitVecVal(2, n)) == BitVecVal(0, n),
            prox['pc'] == BitVecVal(2 ,n),
            prox['x']  == curr['x'],
            prox['y']  == curr['y'],
            prox['z']  == curr['z']
        ),
        And(
        #skip → right
            curr['pc'] == BitVecVal(1 ,n),
            curr['y'] ≠ BitVecVal(0, n),
            URem(curr['y'], BitVecVal(2, n)) == BitVecVal(1, n),
            prox['pc'] == BitVecVal(3 ,n),
            prox['x'] == curr['x'],
            prox['y'] == curr['y'],
            prox['z'] == curr['z']
        ),
        # skip → stop
        And(
            curr['pc'] == BitVecVal(1 ,n),
            curr['y'] == BitVecVal(0, n),
            prox['pc'] == BitVecVal(4 ,n),
            prox['x'] == curr['x'],
            prox['y'] == curr['y'],
            prox['z'] == curr['z']
        ),
        # left → error
        And(
            curr['pc'] == BitVecVal(2 ,n),
            shift_overflow,
            prox['pc'] == BitVecVal(5 ,n),
            prox['x'] == curr['x'],
            prox['y'] == curr['y'],
            prox['z'] == curr['z']
        ),
        # left → Skip
        And(
            curr['pc'] == BitVecVal(2 ,n),
            Not(shift_overflow),
            prox['pc'] == BitVecVal(1 ,n),
            prox['x'] == BitVecVal(2, n) * curr['x'],
            prox['y'] == UDiv(curr['y'], BitVecVal(2, n)),
            prox['z'] == curr['z']
        ),
        # right > error
```

```
            And(
                curr['pc'] == BitVecVal(3 ,n),
                soma_overflow,
                prox['pc'] == BitVecVal(5 ,n),
                prox['x'] == curr['x'],
                prox['y'] == curr['y'],
                prox['z'] == curr['z']
            ),
            # right -> skip
            And(
                curr['pc'] == BitVecVal(3 ,n),
                Not(soma_overflow),
                prox['pc'] == BitVecVal(1 ,n),
                prox['x'] == curr['x'],
                prox['y'] == curr['y'] - BitVecVal(1, n),
                prox['z'] == curr['z'] + curr['x']
            ),
            # stop
            And(
                curr['pc'] == BitVecVal(4 ,n),
                prox['pc'] == BitVecVal(4 ,n),
                prox['x'] == curr['x'],
                prox['y'] == curr['y'],
                prox['z'] == curr['z']
            ),
            # erro
            And(
                curr['pc'] == BitVecVal(5 ,n),
                prox['pc'] == BitVecVal(5 ,n),
                prox['x'] ==curr['x'],
                prox['y'] ==curr['y'],
                prox['z'] ==curr['z']
            )
        )
    )
```

Definimos o invariante x*y+z = a*b e usamos k-indução para verificar que se mantêm verdadeira durante a execução.

In [7]:
```
def invariante(state, n):
    return (state['x'] * state['y']) + state['z'] == (state['a'] *
state['b'])

def kinduction_always(declare, init, trans, inv, k, n):
    solver = Solver()
    states = [declare(i, n) for i in range(k)]

    constraints = []
    constraints.append(init(states[0], n))

    for i in range(k-1):
        constraints.append(trans(states[i], states[i+1], n))
```

```
    inv_falha = [Not(inv(states[i], n)) for i in range(k)]
    constraints.append(Or(inv_falha))

    solver.add(And(constraints))
    if solver.check() == sat:
        print("O caso Base falhou")
        model = solver.model()
        for i, st in enumerate(states):
            pc_value = model.eval(st['pc'])
            x_value = model.eval(st['x'])
            y_value = model.eval(st['y'])
            z_value = model.eval(st['z'])
            print(f"estado {i}: pc = {pc_value}, x = {x_value}, y =
{y_value}, z = {z_value}")
        return


    solver = Solver()

    states_ind = [declare(i, n) for i in range(k + 1)]
    constraints = []

    for i in range(k):
        constraints.append(inv(states_ind[i], n))

    for i in range(k):
        constraints.append(trans(states_ind[i], states_ind[i+1], n))

    constraints.append(Not(inv(states_ind[k], n)))

    solver.add(And(constraints))

    if solver.check() == sat:
        print("O passo indutivo falhou")
        return

    print("(x*y + z = a*b) é um invariante")
    return
```

In [8]:
```
kinduction_always(declare, init, trans, invariante, 5, 10)
```

```
(x*y + z = a*b) é um invariante
```

Para garantir que o nodo Error não é alcançado definimos uma nova função init com as condições adicionais:

a < 2^(n/2);

b < 2^(n/2).

E também definimos um invariante com as condições:

pc != 5;

a < 2^(n/2);

b < 2^(n/2);

x*y + z = a*b.

In [9]:
```
def init_3(curr, n):
    limite = BitVecVal(1 << (n // 2), n)
    return And(
        curr['pc'] == BitVecVal(0, n),
        curr['z'] == BitVecVal(0, n),
        curr['x'] == curr['a'],
        curr['y'] == curr['b'],
        ULT(curr['a'], limite),
        ULT(curr['b'], limite)
    )

def invariante_3(state, n):
    limite = BitVecVal(1 << (n // 2), n)

    return And(
        state['pc'] != BitVecVal(5, n),
        ULT(state['a'], limite),
        ULT(state['b'], limite),
        ZeroExt(n, state['x']) * ZeroExt(n, state['y']) + ZeroExt(n,
state['z']) == ZeroExt(n, state['a']) * ZeroExt(n, state['b'])
    )
```

In [10]:
```
kinduction_always(declare, init_3, trans, invariante_3, 5, 10)
```

```
(x*y + z = a*b) é um invariante
```

Verificamos se o SFOTS chega aos nodos Stop ou Error independente dos valores de a e b. Usamos BMC para verificar se existem loops que o impeçam de chegar a esses nodos.

In [11]:
```
def terminates(state, n):
    #stop = 4, error = 5
    return Or(
        state['pc'] == BitVecVal(4, n),
        state['pc'] == BitVecVal(5, n)
    )


def bmc_eventually(declare, init, trans, prop, bound, n):
    for k in range(1, bound+1):
        solver = Solver()

        states = [declare(i, n) for i in range(k)]

        constraints = []
```

```
        constraints.append(init(states[0], n))

        for i in range(k - 1):
            constraints.append(trans(states[i], states[i + 1], n))

        for st in states:
            constraints.append(Not(prop(st, n)))

        loop = []
        for j in range(k - 1):
            loop.append(trans(states[-1], states[j], n))

        constraints.append(Or(loop))

        solver.add(And(constraints))

        if solver.check() == sat:
            print("Foi encontrado um contraexemplo")
            model = solver.model()

            for i, st in enumerate(states):
                pc_value = model.eval(st['pc'])
                x_value = model.eval(st['x'])
                y_value = model.eval(st['y'])
                z_value = model.eval(st['z'])

                print(f"estado {i}: pc = {pc_value}, "
                      f"x = {x_value}, y = {y_value}, z = {z_value}")

            for j in range(k-1):
                if is_true(model.eval(trans(states[-1], states[j]))):
                    print(f"Loop: de {k-1} para {j}")
                    break

            return False

    print(f"Nenhum contra-exemplo encontrado até {bound}")
    return
```

In [12]:
```
bmc_eventually(declare, init, trans, terminates, 20, 10)
```

```
Nenhum contra-exemplo encontrado até 20
```